SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Code of Conduct:

I Oviya M / 192321117 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *oviya m*

SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task - Answers

COURSE NAME: Software Testing **COURSE CODE:** CSA37

Name: Oviya M **Register No:** 192321117

COURSE OUTCOME COVERED

CO2: Design effective test cases using black-box, white-box testing techniques and to control methods flow.

1. Introduction and Approach

This document answers the CO2_AT2 Reverse Engineering Task (CSA37 Software Testing). In each question only the observable evidence is given (behaviour, traces, logs, a binary or queries). The answers follow the rubric: (1) understand the system, (2) decompose it into components, (3) identify the components and their functions, (4) reconstruct the underlying model (rules, CFG, state machine, decision table) and the test strategy, and (5) document the result with tables and diagrams.

2. Reverse Engineering Method for Testing

Step	Activity	Evidence used
1	Observe inputs and outputs, collect boundaries and error messages	behaviour, logs
2	Group observations into candidate rules and partitions	pattern analysis
3	Decompose into functional components	data flow, responsibilities
4	Infer the testing technique that produced the observed tests	technique signatures
5	Reconstruct model: ECP/BVA tables, CFG, state diagram or decision table	reasoned from evidence
6	Evaluate gaps and design improved tests	coverage criteria

3. Technique Signatures (how to infer the technique)

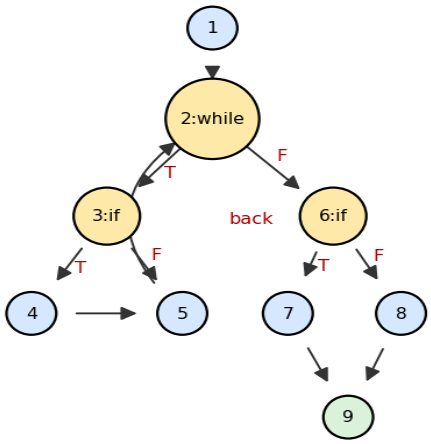
Observed evidence	Technique likely used
Defects at exactly limit values; tests at limit, limit+1	BVA
One test per range of inputs, none at edges	ECP only
Tests change one condition at a time with true/false outcomes	Decision table / MC/DC
Tests follow sequences of events	State transition testing
Coverage report with statement % only	Statement coverage
Odd inputs: long strings, symbols	Error guessing / fuzzing

4. Notation

N = nodes, E = edges, P = predicate nodes, V(G) = E - N + 2 = P + 1. In CFG pictures a yellow circle is a decision node, blue is a statement block and green is an entry/exit node. T and F label true and false edges.

<table>
<tr><th>Sl. No</th><th>Answers</th><th>Bloom's Level</th></tr>
<tr><td>1</td><td>
Observed behaviour (assumed): Withdraw 100 accepted; 99 and 150 rejected (multiples of 100 only); 20,000 accepted; 20,100 "exceeds limit"; amount > balance "insufficient funds"; daily total > 50,000 refused.
Decomposition: Card/PIN validation | amount validation (range, denomination) | balance check | daily-limit check | cash availability | dispense and balance update | receipt/log.
Inferred techniques and adequacy: ECP (valid/invalid amounts) and BVA at 100 and 20,000 are visible; decision-table style checks (balance + limit) appear partly. Gaps: daily cumulative limit, balance exactly equal to amount, ATM cash shortage, PIN retry limit, network failure.
<table>
<tr><th>ID</th><th>Input / condition</th><th>Expected</th></tr>
<tr><td>1</td><td>99 / 100 / 101</td><td>Reject / Accept / Reject (not multiple)</td></tr>
<tr><td>2</td><td>19,900 / 20,000 / 20,100</td><td>Accept / Accept / Reject (limit)</td></tr>
<tr><td>3</td><td>amount = balance; balance + 100</td><td>Accept / Insufficient funds</td></tr>
<tr><td>4</td><td>0, negative, abc</td><td>Reject</td></tr>
<tr><td>5</td><td>two withdrawals making 50,000, then 100</td><td>Last refused (daily limit)</td></tr>
<tr><td>6</td><td>ATM cash < amount</td><td>Cash unavailable message</td></tr>
<tr><td>7</td><td>3 wrong PINs</td><td>Card blocked</td></tr>
</table>
Justification: Every partition, boundary and rule inferred from the behaviour is covered by at least one test.
Detailed explanation: Reverse engineering starts from the observed messages. Rejection of 99 and 150 but acceptance of 100 and 200 shows a minimum and a denomination rule. Rejection at 20,100 but acceptance at 20,000 shows a per-transaction maximum. A different message for amount > balance shows a separate balance check executed after the range check, which defines the order of the checks.
<table>
<tr><th>Observed</th><th>Inferred rule</th><th>Likely technique</th></tr>
<tr><td>99 rejected, 100 accepted</td><td>minimum 100</td><td>BVA</td></tr>
<tr><td>150 rejected</td><td>multiples of 100</td><td>ECP</td></tr>
<tr><td>20,100 rejected</td><td>maximum 20,000</td><td>BVA</td></tr>
<tr><td>amount > balance message</td><td>balance check</td><td>ECP</td></tr>
<tr><td>sixth withdrawal refused</td><td>daily limit</td><td>decision table</td></tr>
</table>
Order of checks: card valid -> PIN valid -> amount format -> amount range -> denomination -> balance -> daily limit -> cash available -> dispense. Test cases must also verify this order (for example an invalid amount with insufficient balance gives the amount error first).
Additional components: Receipt printing, card return, session timeout (30 s no key press) and transaction log: each needs at least one test.
</td><td>5</td></tr>
</table>

2	**Code snippet (from traces):** 1 read n; total=0; i=1 | 2 while(i<=n) | 3 if(i%3==0 || i%5==0) | 4 total+=i | 5 i++ | 6 if(total>100) | 7 print Large | 8 print total | 9 exit	5



Decision points, loop, paths: Decisions: 2 (loop), 3 (compound OR), 6. N=9, E=11, **V(G) = 11 - 9 + 2 = 4**.

Path	Route	Test
P1	1-2-6-8-9	n=0
P2	1-2-3-5-2-6-8-9	n=1
P3	1-2-3-4-5-2-6-8-9	n=3
P4	1-2-3-4-5-2-6-7-9	n=30 (total>100)

Existing test adequacy: Trace with n=10 covers 9 of 11 edges (82% branch): edges 6->7, 7->9 and loop skip never executed.

Reconstructed strategy: P1-P4 (basis paths) + loop tests n=0,1,2,max + MC/DC for (i%3==0 || i%5==0): (F,F), (T,F), (F,T) -> 100% statement, branch and basis path.

Detailed explanation: From the traces the tester can deduce which edges ran. A trace for n=10 visits the loop test, both branches of the if, and the final else print; no trace visits the Large print. This shows the original suite used a single nominal value and therefore achieved decision coverage on only one outcome of the last decision.

Edge	Executed by n=10?	Needed test
1-2	yes	-
2-3 / 2-6	yes	n=0 for loop skip
3-4 / 3-5	yes	-
5-2 back edge	yes	-
6-7 (Large)	NO	n=30
6-8	yes	-

MC/DC for (i%3==0 || i%5==0): (F,F): i=1; (T,F): i=3; (F,T): i=5. These three iterations (reached by n=5) show each condition independently changes the decision, so a single test n=5 plus n=30 gives MC/DC.

Complete strategy: P1-P4 plus n=5 (MC/DC) plus loop boundary n=2 and n=max. Result: 100% statement, 100% branch, all independent paths, MC/DC for the compound condition.

3	**Defect-pattern analysis:** (a) Defects at exact limits (3rd/4th login attempt, transaction limit, session timeout) -> BVA used only partly. (b) Missing field validation (blank, long, special chars) -> ECP weak. (c) Logic flaws in lock-out and failed-transaction paths -> no path/branch testing. (d) Injection and duplicate-submit defects -> found by error guessing. **Decomposition:** Login (credentials, lockout, session) | Transaction (amount, balance, limits, confirmation) | Logout/session timeout | Logging. **Gaps:** Branch coverage of lockout/error paths, concurrent sessions, boundary values for amount and attempts.	5

Component	Technique	Key tests
Login fields	ECP + BVA	username/password length min-1, min, max, max+1; blank; special chars
Attempts	BVA	attempts 2, 3, 4 (lock at 3)
Login CFG	Path testing (V(G)=5)	valid; wrong user; wrong password; locked; expired session
Transaction amount	ECP + BVA	0, 1, limit-1, limit, limit+1, > balance
Session	BVA	timeout-1, timeout, timeout+1

Justification: Combines specification-based (ECP, BVA) and structure-based (path) tests, so each recurring defect pattern has a test aimed at it.

Detailed explanation: Defect logs are analysed by classification. Defects at attempt 3/4 or at the transaction limit point to boundaries; defects with blank or special-character input point to missing equivalence classes; defects only in the lock-out or rollback branch point to code never executed by tests.

Defect log pattern	Inference	Added test
Login allowed on 4th wrong password	BVA missing at attempt limit	attempts 2,3,4
Transaction accepted at limit+1	BVA missing	limit-1, limit, limit+1
Blank username causes error 500	ECP missing blank	blank, space, long
Failed payment leaves balance debited	rollback path untested	path: debit then fail
Two same transactions accepted	duplicate submission not tested	double click test

Components and gaps: Login (credentials, lock-out, session), transaction (amount, balance, limit, rollback), logging. Coverage gaps: error handling paths, concurrency, session expiry boundary.

4

Inferred logic from logs: Failed: stock update to -5, to 1001 (max capacity 1000), non-numeric. Low-stock alert logged at stock 10 and 9 but not at 11 -> reorder level <= 10 (inferred). Success at 0 with "out of stock" alert.

Partitions: Stock after update: <0 invalid | 0 out of stock | 1-10 low-stock alert | 11-1000 normal | >1000 invalid. Update quantity: removal > available (invalid), 0, valid, non-numeric.

Decomposition: Input validator | stock calculator | alert generator | transaction logger.

Test	Expected	Gap it covers
Stock result -1, 0, 1	Reject, alert, alert	lower boundary
Stock result 10, 11	Alert, no alert	threshold boundary
Stock result 1000, 1001	Accept, Reject	capacity boundary
Remove more than available	Reject, stock unchanged	negative stock
Two simultaneous updates	Consistent stock	concurrency (log gap)
Decimal / text quantity	Reject	format

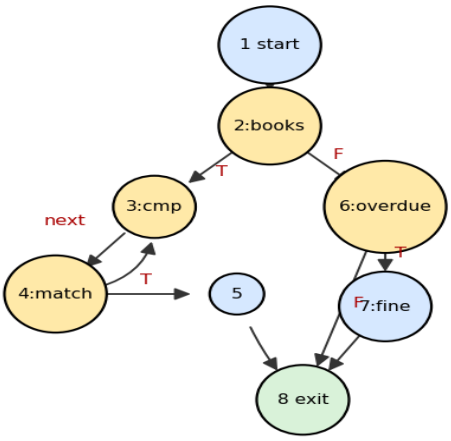
Justification: Each test is derived from an inferred boundary or a failure seen in the logs, so the hidden rules are verified and gaps in the logs are closed.

Detailed explanation: The logs give pairs (input, result). Sorting the pairs by input shows the thresholds where the result changes: 10/11 for the alert, 1000/1001 for capacity, -1/0 for negative stock. These changes are the boundary values to test.

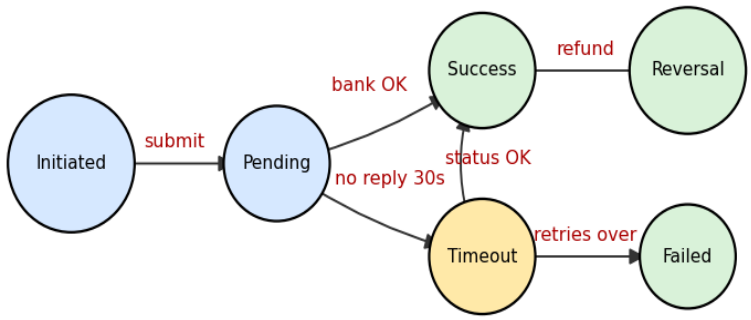
Log entry	Result	Inferred boundary
update stock to 11	no alert	above reorder level
update stock to 10	alert	reorder level 10
update stock to 0	alert out-of-stock	0 is valid
update stock to -1	failed	min 0
update stock to 1000	success	max capacity
update stock to 1001	failed	max 1000

Hidden gaps: Logs contain no entry for exactly equal removal (stock 5, remove 5), repeated update of the same item, or alert de-duplication; tests for these are added.

5

5	**Mapping profile to logic:** CPU spike on search/catalog load -> nested loops (books x members); steadily rising memory in bulk issue/return -> loop allocating without release; long flat 100% CPU -> possible tight/unbounded loop; periodic spikes -> decision clusters (overdue and fine calculation for each record). **Conceptual CFG:** Search module inferred: 1 start | 2 loop over books | 3 compare | 4 match? | 5 add result | 6 overdue check | 7 fine calculation | 8 exit.  **Path testing strategy:** V(G) = P + 1 = 6 basis paths. Focus on resource-heavy paths: loops with 0, 1, N and 100,000 records; matching vs no match; overdue fine path for all records; repeated issue/return (memory growth). **Impact on stability:** Each path is run under stress while CPU/memory are monitored with limits. Detects O(n^2) loops, leaks and non-terminating loops, so response time and crash risk are controlled. **Detailed explanation:** CPU and memory spikes are executed code made visible. A CPU spike that grows with the number of books means a loop over the collection; a quadratic growth means nested loops. A plateau that does not return to idle suggests a wait loop or busy loop. A memory ramp that never returns suggests allocation inside a loop without release. | Profile observation | Logic inferred | Test | | --- | --- | --- | | CPU spike at search, grows with N | loop over books | N = 0, 1, 1000, 100000 | | CPU grows with N squared | nested loop compare | N = 10000 timing | | Memory rises per issue | allocation in loop | repeat issue 10,000 times | | Periodic CPU peaks | per-record overdue check | all overdue / none overdue | **Stability impact:** Path tests with monitored resources give a pass/fail limit (e.g. response time under 2 s, memory flat after test) and identify the paths that threaten system stability under load.	6

6	**State machine reconstructed:** States: Initiated, Pending, Success, Timeout, Reversal, Failed. Triggers: submit, bank OK, no reply in 30 s, status check, refund.	5



Technique identified: State Transition Testing (state table with 0-switch and 1-switch coverage).

ID	From	Event	To / Result
L1	Initiated	submit	Pending
L2	Pending	bank OK	Success
L3	Pending	no reply 30s	Timeout
L4	Timeout	status OK	Success
L5	Timeout	retries exhausted	Failed
L6	Success	refund	Reversal
I1	Success	pending/timeout event	Rejected (409)
I2	Reversal	bank OK	Rejected
I3	Pending	refund	Rejected (nothing to reverse)
I4	Failed	bank OK	Rejected

Coverage: All 6 legal and 4 illegal transitions, plus sequence Pending -> Timeout -> Success -> Reversal (1-switch).

Detailed explanation: Each JSON response contains a state and a trigger. Collecting them yields a state table. Missing combinations in the observed data indicate transitions that are either illegal or untested. The test suite must include both because payment systems must reject illegal changes (for example Success to Pending).

State \ Event	submit	bank OK	no reply	status OK	refund
Initiated	Pending	-	-	-	-
Pending	-	Success	Timeout	-	reject
Timeout	-	-	-	Success / Failed	reject
Success	-	reject	-	-	Reversal
Reversal	-	reject	-	-	reject

Coverage levels: 0-switch: all 6 legal transitions; illegal transitions: 10 pairs above marked reject; 1-switch: Pending-Timeout-Success-Reversal and Pending-Timeout-Failed.

7

Fuzzing observations (assumed): Messages up to 256 bytes accepted, 257 rejected; empty rejected; key sizes accepted only 16/24/32 bytes; control characters and null bytes rejected.

Decomposition: Input validation (length, charset, format) | key handling | encryption engine | output encoding.

Parameter	BVA / test values	Expected
Message length	0, 1, 255, 256, 257, 10,000	Reject, Accept, Accept, Accept, Reject, Reject
Key length	15, 16, 17, 23, 24, 25, 31, 32, 33	Only 16/24/32 accepted
Multi-byte chars	255 bytes + 2-byte char	Reject (byte length 257)
Injection strings	' OR 1=1 --, <script>, ../, %00, %n	Rejected/escaped

Justification: Testing at buffer size, +1 and very large inputs proves lengths are checked before copying (buffer overflow), and injection payloads confirm validation/encoding, so malformed input cannot reach the encryption engine.

Detailed explanation: Fuzzing means sending many varied inputs and recording which are accepted. The accepted/rejected pattern shows the validation rules. Rejections occurring exactly at length 257 show a 256-byte limit; rejections of keys with 15, 17 or 20 bytes show a discrete rule (16/24/32 bytes); rejections of %00 show null-byte filtering.

Fuzz input	Observed	Inference
256 'A'	accepted	max 256
257 'A'	rejected	limit exactly 256
key 20 bytes	rejected	key sizes discrete
'<script>'	accepted but escaped	output encoding present
'%00'	rejected	null byte filter

Security argument: If the length check is missing or placed after copying, input longer than the buffer overwrites memory. BVA at the buffer size, plus very large inputs, proves the check happens first. Injection payloads prove that data is treated as data and never as commands.

5

8

Original approach and inadequacy: Only statement coverage was used: a single test (young, claims, smoker) executes every line but never makes decisions false and never tests conditions separately.

Code: 1 premium=base | 2 if(age<25 && claims>0) | 3 premium*=1.5 | 4 if(smoker || age>60) | 5 premium*=1.2 | 6 return.

CFG and paths: N=6, E=7, V(G)=3; all 4 combinations of the two decisions are paths (no loop) -> complete path coverage = 4 tests (base 1000):

Path	Data	Expected premium
P1 T,T	age 20, claims 1, smoker	1800
P2 F,F	age 40, claims 0, non-smoker	1000
P3 T,F	age 20, claims 1, non-smoker	1500
P4 F,T	age 65, claims 0, non-smoker	1200
MC/DC extra	age 20, claims 0, non-smoker (B)	1000
MC/DC extra	age 40, claims 1 (A)	1000
MC/DC extra	age 40, smoker (C)	1200

Result: 100% branch and path coverage; each condition (age<25, claims>0, smoker, age>60) is shown to affect its decision, so && / || and comparison errors in premium calculation are caught.

Detailed explanation: 100% statement coverage is achieved by one test with every if true. The legacy suite therefore never made any decision false. Multi-condition decisions need condition or MC/DC coverage to show that each condition matters.

Criterion	Legacy suite	Reconstructed suite
Statement	100%	100%
Branch	50% (no false outcomes)	100%
Condition	25%	100%
Path (4 paths)	25%	100%
MC/DC	0	100%

Risk reduction: Errors such as wrong multiplier, || instead of &&, or wrong age threshold produce a wrong premium only for specific customers; the seven tests make each of those faults produce a different premium from the expected one.

5

<table>
<tr>
<td>9</td>
<td>
Defect patterns: Lost files for very large files (above 2 GB), zero-byte and special-name files, and slow or interrupted networks. This suggests developers tested only small/medium files on a fast network.
Inferred ECP of developers: Size: small, medium. Type: common documents. Network: fast, stable. Missing: empty, huge, locked/temp files, slow/offline network.
Revised strategy
• BVA on size: 0, 1 byte, chunk size -1/ = / +1 (e.g. 4 MB), 2 GB -1, 2 GB, 2 GB +1.
• ECP on type: documents, media, executables, locked/open files, names with special characters.
• Network: fast, throttled (56 kbps), intermittent, drop mid-transfer, resume.
• Stress: many parallel syncs, low disk, 100,000 small files.
Justification: Sync must verify checksums before local delete and commit atomically; these tests force failure at each boundary, so no path exists where a file is deleted before safe upload, preventing data loss.
Detailed explanation: Defect reports are grouped by file size, type and network speed. If all lost files are larger than about 2 GB the developers' classes probably stopped at large; if lost files all have special characters in names the file-name class was missing; if lost files occur only when speed is under a threshold, timeouts delete local copies before upload completes.
<table>
<tr><th>Defect pattern</th><th>Developers' probable ECP</th><th>Missing class</th></tr>
<tr><td>lost files > 2 GB</td><td>small, medium, large</td><td>huge (> 2 GB)</td></tr>
<tr><td>lost zero-byte files</td><td>none</td><td>empty file</td></tr>
<tr><td>lost on slow network</td><td>normal speed</td><td>throttled, intermittent</td></tr>
<tr><td>lost when file open</td><td>closed files</td><td>locked files</td></tr>
</table>
Stress and BVA: Chunk boundary tests (4 MB -1, 4 MB, 4 MB +1), 2 GB boundaries, and 100 parallel uploads on a throttled network, plus kill-the-connection tests at 50% upload. Expected: file remains on the source until checksum is verified.
</td>
<td>5</td>
</tr>
</table>

10

Inferred logic from SQL: Pattern: WHERE category='X' AND price BETWEEN a AND b AND brand IN (...) AND NOT out_of_stock. So: different filters are combined with AND, multiple values of one filter with OR (IN), and out-of-stock excluded with NOT.

Conditions: C1 Category, C2 Price range, C3 Brand (1 or more), C4 In-stock only. 2^4 = 16 rules; Y = filter applied.

Rule	C1	C2	C3	C4	Expected WHERE clause
1	N	N	N	N	none (all products)
2	N	N	N	Y	NOT out_of_stock
3	N	N	Y	N	brand IN
4	N	N	Y	Y	brand IN AND in stock
5	N	Y	N	N	price BETWEEN
6	N	Y	N	Y	price AND in stock
7	N	Y	Y	N	price AND brand
8	N	Y	Y	Y	price AND brand AND stock
9	Y	N	N	N	category
10	Y	N	N	Y	category AND stock
11	Y	N	Y	N	category AND brand
12	Y	N	Y	Y	category AND brand AND stock
13	Y	Y	N	N	category AND price
14	Y	Y	N	Y	category AND price AND stock
15	Y	Y	Y	N	category AND price AND brand
16	Y	Y	Y	Y	all four

Evaluation of completeness: All 16 combinations are covered. Not covered by this model: price boundaries (min = max, min > max), empty results, multi-brand OR logic, sorting/pagination, special characters (SQL injection); these are added as BVA and negative tests (about 8 extra), so the model is complete for filter logic and extendable.

Detailed explanation: SQL queries show which filters the user selected and how the module joined them. Comparing queries with the same filters but different results shows the NOT behaviour. The logic is therefore reconstructed as: category AND price range AND (brand1 OR brand2 ...) AND NOT out of stock.

Observed query fragment	Meaning
category = 'Shoes'	single value filter
price BETWEEN 1000 AND 3000	inclusive range
brand IN ('Nike','Adidas')	OR across brands
AND stock > 0	NOT out of stock
ORDER BY price	sorting, outside filter logic

Completeness evaluation: 16/16 filter combinations covered; additional tests for single vs multiple brands, price boundaries (min = max, min > max), empty results and injection characters bring total tests to about 24 and make the model complete for search behaviour.

5

5. Summary of Reconstructions

Q	System	Inferred model	Technique reconstructed
1	ATM withdrawal	limits, multiples of 100, balance, daily limit	ECP + BVA + decision table
2	Code with traces	CFG, V(G)=4	Basis path + MC/DC
3	Login and transactions	defect pattern map	ECP, BVA, path
4	Inventory logs	reorder level 10, capacity 1000	ECP + BVA
5	Library .exe profile	nested loops, decision clusters	Path testing + stress
6	Payment gateway	6-state machine	State transition
7	Encrypted module	length 256, key 16/24/32	BVA + injection tests
8	Premium calculator	two compound decisions, V(G)=3	Branch + path + MC/DC
9	File sync	size/type/network classes	ECP + BVA + stress
10	Search and filter SQL	AND/OR/NOT filter logic, 16 rules	Decision table

6. Overall Conclusion

Reverse engineering a test approach starts with evidence: failures at edges reveal boundaries, repeated sequences reveal states, and coverage figures reveal the criteria used. By turning this evidence into explicit models (partitions, CFGs, state tables and decision tables) the hidden logic of each system was reconstructed and its weaknesses became visible. The reconstructed test suites are systematic, traceable and complete with respect to the inferred model, and they provide a documented basis for further testing even when no source code or test documentation exists.

7. Limitations and Assumptions

Because only partial information was supplied, concrete numbers (limits, thresholds, sizes) are inferred assumptions based on typical systems and should be confirmed with stakeholders. The reconstructed models are hypotheses that are refined by running the proposed tests and comparing the actual behaviour with the inferred behaviour.

Appendix A: Test Case Specification - ATM Withdrawal (Q1)

ID	Precondition	Input	Expected result	Technique
A01	Valid card, PIN ok, balance 50,000	withdraw 100	Dispensed, balance 49,900	BVA min
A02	same	withdraw 99	Invalid amount message	BVA min-1
A03	same	withdraw 101	Multiples of 100 message	ECP
A04	same	withdraw 150	Multiples of 100 message	ECP
A05	same	withdraw 20,000	Dispensed	BVA max
A06	same	withdraw 20,100	Limit exceeded message	BVA max+1
A07	balance 500	withdraw 500	Dispensed, balance 0	BVA balance
A08	balance 500	withdraw 600	Insufficient funds	ECP
A09	balance 500	withdraw 0	Invalid amount	ECP
A10	any	withdraw -100	Invalid amount	ECP
A11	any	withdraw abc	Invalid input	ECP
A12	today's total 50,000	withdraw 100	Daily limit message	decision table
A13	ATM cash 300	withdraw 500	Cash unavailable	decision table
A14	valid card	3 wrong PIN	Card retained/blocked	ECP
A15	valid card	no key press 30 s	Session timeout, card returned	BVA time

Appendix B: State-Based Test Suite - Payment Gateway (Q6)

ID	Start state	Event	Expected state	Result
S01	Initiated	submit	Pending	pass
S02	Pending	bank OK	Success	pass
S03	Pending	timeout 30 s	Timeout	pass
S04	Timeout	status check OK	Success	pass
S05	Timeout	retries exhausted	Failed	pass
S06	Success	refund	Reversal	pass
S07	Success	bank OK again	Success (idempotent), no double credit	reject
S08	Success	timeout event	Success, event ignored	reject
S09	Reversal	bank OK	Reversal	reject
S10	Failed	bank OK	Failed	reject
S11	Pending	refund	Pending, error 409	reject
S12	Initiated	refund	Initiated, error	reject
S13	Pending-Timeout-Success-Reversal	sequence	each transition valid	1-switch
S14	Pending-Timeout-Failed	sequence	each transition valid	1-switch

Appendix C: BVA and Negative Tests - Encrypted Module (Q7)

ID	Field	Input	Expected
C01	message	empty (0 bytes)	Rejected
C02	message	1 byte	Encrypted
C03	message	255 bytes	Encrypted
C04	message	256 bytes	Encrypted
C05	message	257 bytes	Rejected, no overflow
C06	message	1 MB	Rejected, no crash
C07	key	15, 17 bytes	Rejected
C08	key	16, 24, 32 bytes	Accepted
C09	key	33 bytes	Rejected
C10	message	' OR 1=1 --	Treated as data
C11	message	<script>alert(1)</script>	Escaped / rejected

ID	Field	Input	Expected
C12	message	../../etc/passwd	Rejected
C13	message	null byte %00	Rejected
C14	message	format string %s%s%n	Treated as data

Appendix D: Traceability Matrix

Question	Requirement / evidence	Test IDs	Coverage measure
Q1	Withdrawal limits, balance, daily limit	A01-A15	partitions + boundaries 100%
Q2	CFG paths and MC/DC	P1-P4, n=5	branch 100%, paths 4/4
Q3	Login and transaction defects	table Q3	each defect pattern has a test
Q4	Stock thresholds	table Q4	boundaries 100%
Q5	CPU/memory paths	table Q5	paths monitored
Q6	State machine	S01-S14	transitions 6/6 legal, 4/4 illegal
Q7	Validation and injection	C01-C14	boundaries + payloads
Q8	Premium conditions	7 tests	branch, path, MC/DC 100%
Q9	File sync classes	table Q9	classes + boundaries
Q10	Filter combinations	16 rules	rules 100%

Appendix E: Glossary

Term	Meaning
Reverse engineering	Deriving design and logic from observed behaviour or artefacts
Equivalence class	Set of inputs expected to be processed identically
Boundary value	Value at or adjacent to the edge of a class
CFG	Graph of possible execution flow
Basis path	Independent path used to build the minimal path set
State transition	Change of state caused by an event
Decision table	Table of condition combinations and actions
Fuzzing	Automated random or semi-random input generation
MC/DC	Each condition independently affects the decision

Appendix F: Decision Table Test Data - Search and Filter (Q10)

Filters: C1 category = Shoes, C2 price 1000-3000, C3 brand in (Nike, Adidas), C4 in-stock only. Expected result is the set of products satisfying all selected filters.

Rule	C1	C2	C3	C4	Test data	Expected result
1	N	N	N	N	no filter	all products
2	N	N	N	Y	in-stock only	products with stock > 0
3	N	N	Y	N	Nike, Adidas	all Nike or Adidas items
4	N	N	Y	Y	Nike, in stock	Nike items with stock
5	N	Y	N	N	1000-3000	price in range
6	N	Y	N	Y	1000-3000, in stock	range and stock
7	N	Y	Y	N	Adidas, 1000-3000	brand and range
8	N	Y	Y	Y	Adidas, range, stock	brand, range, stock
9	Y	N	N	N	Shoes	all shoes
10	Y	N	N	Y	Shoes, in stock	shoes with stock
11	Y	N	Y	N	Shoes, Nike	Nike shoes
12	Y	N	Y	Y	Shoes, Nike, stock	Nike shoes in stock
13	Y	Y	N	N	Shoes, 1000-3000	shoes in range
14	Y	Y	N	Y	Shoes, range, stock	shoes range stock
15	Y	Y	Y	N	Shoes, range, Nike	Nike shoes in range
16	Y	Y	Y	Y	all filters	intersection of all

Additional Extension Tests for Q10

ID	Test	Expected
E1	price min = max (2000-2000)	only items priced 2000
E2	price min > max (3000-1000)	validation message or empty list
E3	brand with no products	empty result message, no error
E4	search text ' OR 1=1 --	treated as text, no SQL injection
E5	10,000 matching items	pagination correct
E6	price 999 / 1000 / 3000 / 3001	excluded / included / included / excluded

Appendix G: Class and Boundary Specification - Inventory (Q4) and File Sync (Q9)

ID	Module	Input	Expected
G01	Inventory	stock result -1	Update fails
G02	Inventory	stock result 0	Success + out-of-stock alert
G03	Inventory	stock result 1	Success + low-stock alert
G04	Inventory	stock result 10	Low-stock alert
G05	Inventory	stock result 11	Success, no alert
G06	Inventory	stock result 1000	Success
G07	Inventory	stock result 1001	Update fails
G08	Inventory	remove > available	Update fails, stock unchanged
G09	Sync	file 0 bytes	Synced, checksum ok
G10	Sync	file 1 byte	Synced
G11	Sync	4 MB - 1, 4 MB, 4 MB + 1	Synced, all chunks verified
G12	Sync	2 GB - 1, 2 GB, 2 GB + 1	Synced, no truncation
G13	Sync	file locked by another program	Retry later, not deleted
G14	Sync	connection drops at 50%	Resume, original kept until verified
G15	Sync	56 kbps throttled network	Completes or retries, no loss
G16	Sync	100 parallel uploads	All complete, counts match

Appendix H: Reconstruction Review Checklist

Check	Question to ask	Applies to
Evidence	Is every inferred rule supported by at least one observation or log entry?	Q1, Q3, Q4, Q9
Boundaries	Is each inferred limit tested at limit-1, limit and limit+1?	Q1, Q4, Q7, Q9
Completeness	Are all 2^n combinations or all state-event pairs covered?	Q6, Q10
Paths	Does V(G) equal the number of independent paths designed?	Q2, Q5, Q8
Negative tests	Are invalid inputs, illegal transitions and attack strings included?	Q6, Q7
Resources	Were CPU, memory and time measured per path?	Q5
Documentation	Does each model have a table or diagram and a test list?	all
Assumptions	Are inferred numbers marked for confirmation with the owner?	all

Appendix I: Technique Comparison Used in the Reconstructions

Technique	Strength	Limitation	Used in
ECP	Few tests, broad coverage	Misses edge faults	Q1, Q4, Q9
BVA	Finds off-by-one and limit faults	Needs ordered ranges	Q1, Q4, Q7, Q9
Decision table	Complete combinations	Grows as 2^n	Q10
State transition	Sequences and illegal events	Needs state model	Q6
Path / basis path	Structural completeness	Needs code or traces	Q2, Q5, Q8
Fuzzing	Finds unexpected inputs	No expected oracle	Q7

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation